下载APP

09 | 所有权：一个值可以有多个所有者么？

2021-09-10 陈天

《陈天 · Rust 编程第一课》 课程介绍 >



讲述：陈天
时长 15:35 大小 14.28M

你好，我是陈天。

之前介绍的单一所有权规则，能满足我们大部分场景中分配和使用内存的需求，而且在编译时，通过 Rust 借用检查器就能完成静态检查，不会影响运行时效率。

但是，规则总会有例外，在日常工作中有些特殊情况该怎么处理呢？

> 一个有向无环图（DAG）中，某个节点可能有两个以上的节点指向它，这个按照所有权模型怎么表述？
>
> 多个线程要访问同一块共享内存，怎么办？

我们知道，这些问题在程序运行过程中才会遇到，在编译期，所有权的静态检查无法处理它们，所以为了更好的灵活性，Rust 提供了**运行时的动态检查**，来满足特殊场景下的需求。

这也是 Rust 处理很多问题的思路：编译时，处理大部分使用场景，保证安全性和效率；运行时，处理无法在编译时处理的场景，会牺牲一部分效率，提高灵活性。后续讲到静态分发和动态分发也会有体现，这个思路很值得我们借鉴。

那具体如何在运行时做动态检查呢？运行时的动态检查又如何与编译时的静态检查自洽呢？

Rust 的答案是使用引用计数的智能指针：**Rc（Reference counter）和 Arc（Atomic reference counter）**。这里要特别说明一下，Arc 和 ObjC/Swift 里的 ARC（Automatic Reference Counting）不是一个意思，不过它们解决问题的手段类似，都是通过引用计数完成的。

Rc

我们先看 Rc。对某个数据结构 T，我们可以创建引用计数 Rc，使其有多个所有者。Rc 会把对应的数据结构创建在堆上，我们在第二讲谈到过，堆是唯一可以让动态创建的数据被到处使用的内存。

复制代码

```
use std::rc::Rc;
fn main() {
  let a = Rc::new(1);
}
```

之后，如果想对数据创建更多的所有者，我们可以通过 clone() 来完成。

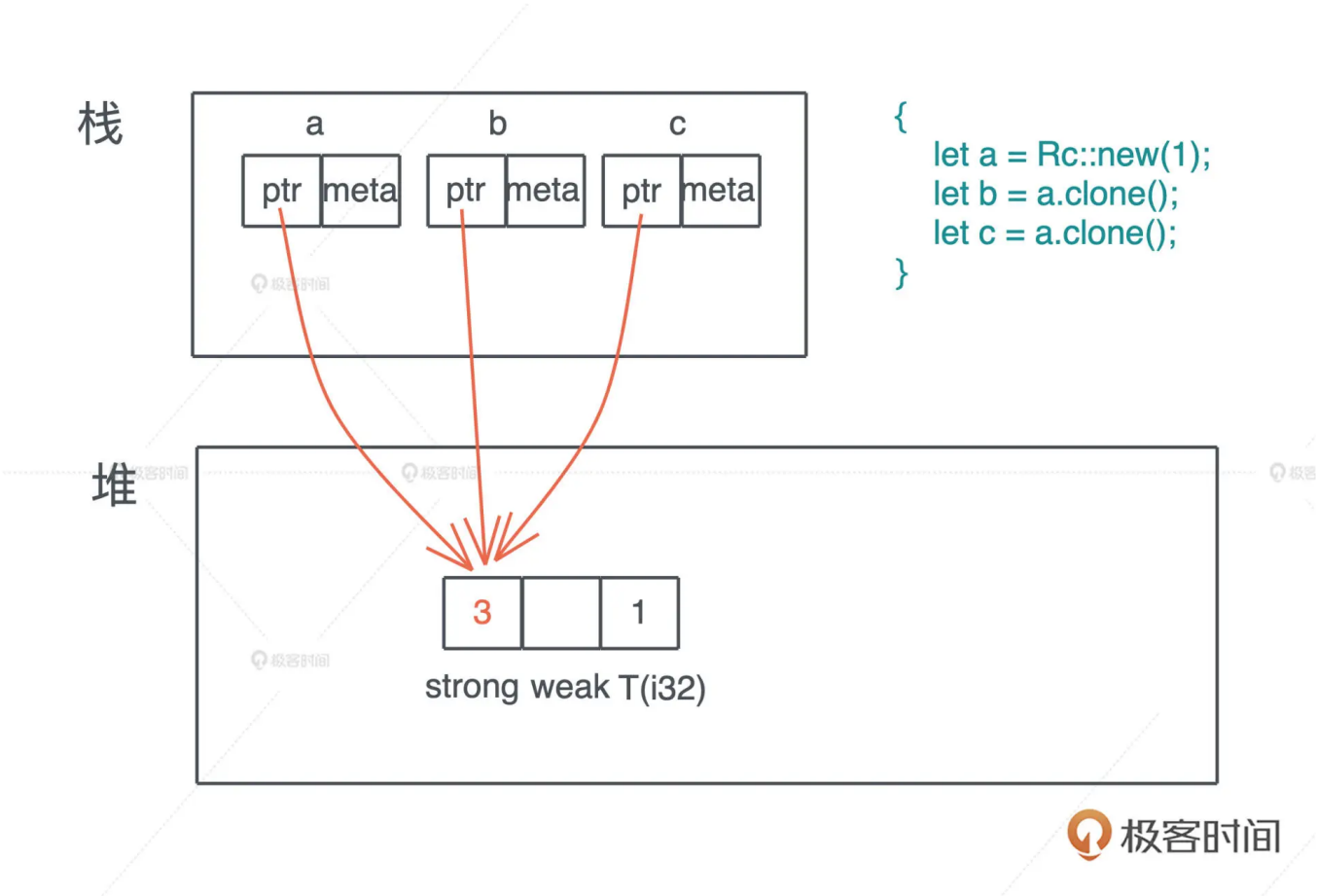
对一个 Rc 结构进行 clone()，不会将其内部的数据复制，只会增加引用计数。而当一个 Rc 结构离开作用域被 drop() 时，也只会减少其引用计数，直到引用计数为零，才会真正清除对应的内存。

复制代码

```
use std::rc::Rc;
```

```
fn main() {
    let a = Rc::new(1);
    let b = a.clone();
    let c = a.clone();
}
```

上面的代码我们创建了三个 Rc，分别是 a、b 和 c。它们共同指向堆上相同的数据，也就是说，堆上的数据有了三个共享的所有者。在这段代码结束时，c 先 drop，引用计数变成 2，然后 b drop、a drop，引用计数归零，堆上内存被释放。



你也许会有疑问：为什么我们生成了对同一块内存的多个所有者，但是，编译器不抱怨所有权冲突呢？

仔细看这段代码：首先 a 是 Rc::new(1) 的所有者，这毋庸置疑；然后 b 和 c 都调用了 a.clone()，分别得到了一个新的 Rc，所以从编译器的角度，abc 都各自拥有一个 Rc。如果文字你觉得稍微有点绕，看看 Rc 的 clone() 函数的实现，就很清楚了（源代码）：

复制代码

```
fn clone(&self) -> Rc<T> {

```

```
        // 增加引用计数
        self.inner().inc_strong();
        // 通过 self.ptr 生成一个新的 Rc 结构
        Self::from_inner(self.ptr)
    }
```

所以，Rc 的 clone() 正如我们刚才说的，不复制实际的数据，只是一个引用计数的增加。

你可能继续会疑惑：Rc 是怎么产生在堆上的？并且为什么这段堆内存不受栈内存生命周期的控制呢？

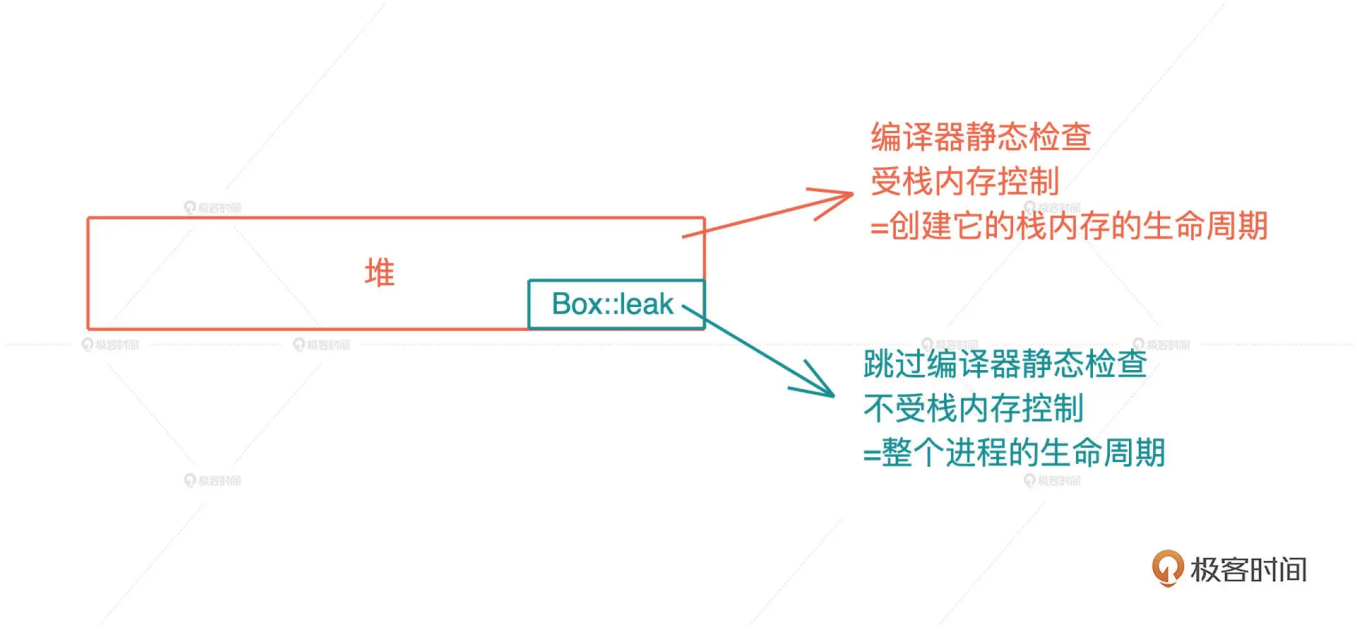
Box::leak() 机制

上一讲我们讲到，在所有权模型下，堆内存的生命周期，和创建它的栈内存的生命周期保持一致。所以 Rc 的实现似乎与此格格不入。的确，如果完全按照上一讲的单一所有权模型，Rust 是无法处理 Rc 这样的引用计数的。

Rust 必须提供一种机制，让代码可以像 C/C++ 那样，**创建不受栈内存控制的堆内存**，从而绕过编译时的所有权规则。Rust 提供的方式是 Box::leak()。

Box 是 Rust 下的智能指针，它可以强制把任何数据结构创建在堆上，然后在栈上放一个指针指向这个数据结构，但此时堆内存的生命周期仍然是受控的，跟栈上的指针一致。我们后续讲到智能指针时会详细介绍 Box。

Box::leak()，顾名思义，它创建的对象，从堆内存上泄漏出去，不受栈内存控制，是一个自由的、生命周期可以大到和整个进程的生命周期一致的对象。



所以我们相当于主动撕开了一个口子，允许内存泄漏。注意，在 C/C++ 下，其实你通过 malloc 分配的每一片堆内存，都类似 Rust 下的 Box::leak()。我很喜欢 Rust 这样的设计，它符合最小权限原则（Principle of least privilege），最大程度帮助开发者撰写安全的代码。

有了 Box::leak()，我们就可以跳出 Rust 编译器的静态检查，保证 Rc 指向的堆内存，有最大的生命周期，然后我们再通过引用计数，在合适的时机，结束这段内存的生命周期。如果你对此感兴趣，可以看 Rc::new() 的源码。

插一句，在学习语言的过程中，不要因为觉得自己是个初学者，就不敢翻阅标准库的源码，相反，遇到不懂的地方，如果你去看对应的源码，得到的是第一手的知识，一旦搞明白，就会学得非常扎实，受益无穷。

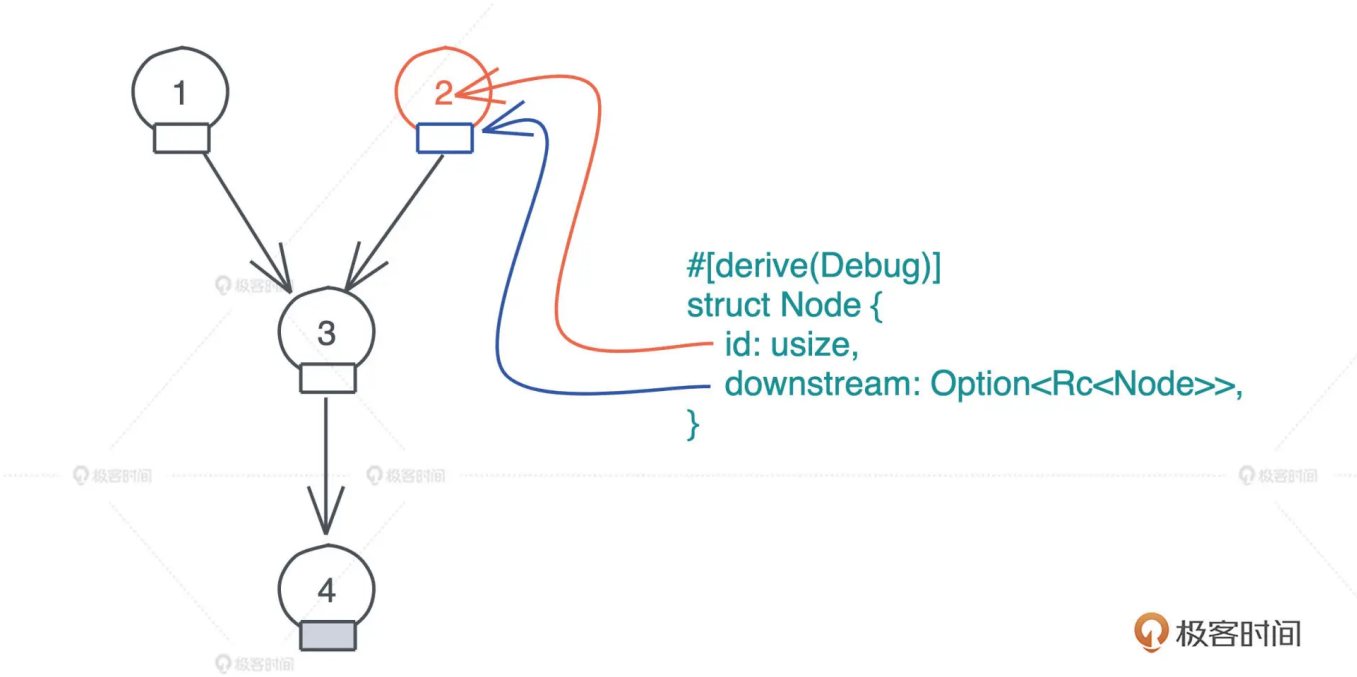
搞明白了 Rc，我们就进一步理解 Rust 是如何进行所有权的静态检查和动态检查了：

> 静态检查，靠编译器保证代码符合所有权规则；
>
> 动态检查，通过 Box::leak 让堆内存拥有不受限的生命周期，然后在运行过程中，通过对引用计数的检查，保证这样的堆内存最终会得到释放。

实现 DAG

现在我们用 Rc 来实现之前无法实现的 DAG。

假设 Node 就只包含 id 和指向下游（downstream）的指针，因为 DAG 中的一个节点可能被多个其它节点指向，所以我们使用 `Rc<Node>` 来表述它；一个节点可能没有下游节点，所以我们用 `Option<Rc<Node>>` 来表述它。



要建立这样一个 DAG，我们需要为 Node 提供以下方法：

new()：建立一个新的 Node。

update_downstream()：设置 Node 的 downstream。

get_downstream()：clone 一份 Node 里的 downstream。

有了这些方法，我们就可以创建出拥有上图关系的 DAG 了（代码 1）：

复制代码

```
use std::rc::Rc;

#[derive(Debug)]
struct Node {
    id: usize,
    downstream: Option<Rc<Node>>,
}

impl Node {
    pub fn new(id: usize) -> Self {
        Self {
            id,
```

```
            downstream: None,
        }
    }

    pub fn update_downstream(&mut self, downstream: Rc<Node>) {
        self.downstream = Some(downstream);
    }

    pub fn get_downstream(&self) -> Option<Rc<Node>> {
        self.downstream.as_ref().map(|v| v.clone())
    }
}

fn main() {
    let mut node1 = Node::new(1);
    let mut node2 = Node::new(2);
    let mut node3 = Node::new(3);
    let node4 = Node::new(4);
    node3.update_downstream(Rc::new(node4));

    node1.update_downstream(Rc::new(node3));
    node2.update_downstream(node1.get_downstream().unwrap());
    println!("node1: {:?}, node2: {:?}", node1, node2);
}
```

RefCell

在运行上述代码时，细心的你也许会疑惑：整个 DAG 在创建完成后还能修改么？

按最简单的写法，我们可以在上面的代码 1 的 `main()` 函数后，加入这段代码（代码 2），来修改 Node3 使其指向一个新的节点 Node5：

复制代码

```
let node5 = Node::new(5);
let node3 = node1.get_downstream().unwrap();
node3.update_downstream(Rc::new(node5));

println!("node1: {:?}, node2: {:?}", node1, node2);
```

然而，它无法编译通过，编译器会告诉你“node3 cannot borrow as mutable”。

这是因为 **Rc 是一个只读的引用计数器**，你无法拿到 Rc 结构内部数据的可变引用，来修改这个数据。这可怎么办？

这里，我们需要使用 RefCell。

和 Rc 类似，RefCell 也绕过了 Rust 编译器的静态检查，允许我们在运行时，对某个只读数据进行可变借用。这就涉及 Rust 另一个比较独特且有点难懂的概念：内部可变性（interior mutability）。

内部可变性

有内部可变性，自然能联想到外部可变性，所以我们先看这个更简单的定义，对比着学。

当我们用 `let mut` 显式地声明一个可变的值，或者，用 `&mut` 声明一个可变引用时，编译器可以在编译时进行严格地检查，保证只有可变的值或者可变的引用，才能修改值内部的数据，这被称作外部可变性（exterior mutability），外部可变性通过 `mut` 关键字声明。

然而，这样不够灵活，有时候我们希望能够绕开这个编译时的检查，对并未声明成 `mut` 的值或者引用，也想进行修改。也就是说，**在编译器的眼里，值是只读的，但是在运行时，这个值可以得到可变借用，从而修改内部的数据**，这就是 `RefCell` 的用武之地。

我们看一个简单的例子（代码 2）：

复制代码

```
use std::cell::RefCell;

fn main() {
    let data = RefCell::new(1);
    {
        // 获得 RefCell 内部数据的可变借用
        let mut v = data.borrow_mut();
        *v += 1;
    }
    println!("data: {:?}", data.borrow());
}
```

在这个例子里，data 是一个 RefCell，其初始值为 1。可以看到，我们并未将 data 声明为可变变量。之后我们可以通过使用 RefCell 的 `borrow_mut()` 方法，来获得一个可变的内

部引用，然后对它做加 1 的操作。最后，我们可以通过 RefCell 的 `borrow()` 方法，获得一个不可变的内部引用，因为加了 1，此时它的值为 2。

你也许奇怪，这里为什么要把获取和操作可变借用的两句代码，用花括号分装到一个作用域下？

因为根据所有权规则，在同一个作用域下，我们**不能同时有活跃的可变借用和不可变借用**。通过这对花括号，我们明确地缩小了可变借用的生命周期，不至于和后续的不可变借用冲突。

这里再想一步，如果没有这对花括号，这段代码是无法编译通过？还是运行时会出错（代码 3）？

复制代码

```
use std::cell::RefCell;

fn main() {
    let data = RefCell::new(1);

    let mut v = data.borrow_mut();
    *v += 1;

    println!("data: {:?}", data.borrow());
}
```

如果你运行代码 3，编译没有任何问题，但在运行到第 9 行时，会得到：“already mutably borrowed: BorrowError” 这样的错误。可以看到，所有权的借用规则在此依旧有效，只不过它在运行时检测。

这就是外部可变性和内部可变性的重要区别，我们用下表来总结一下：

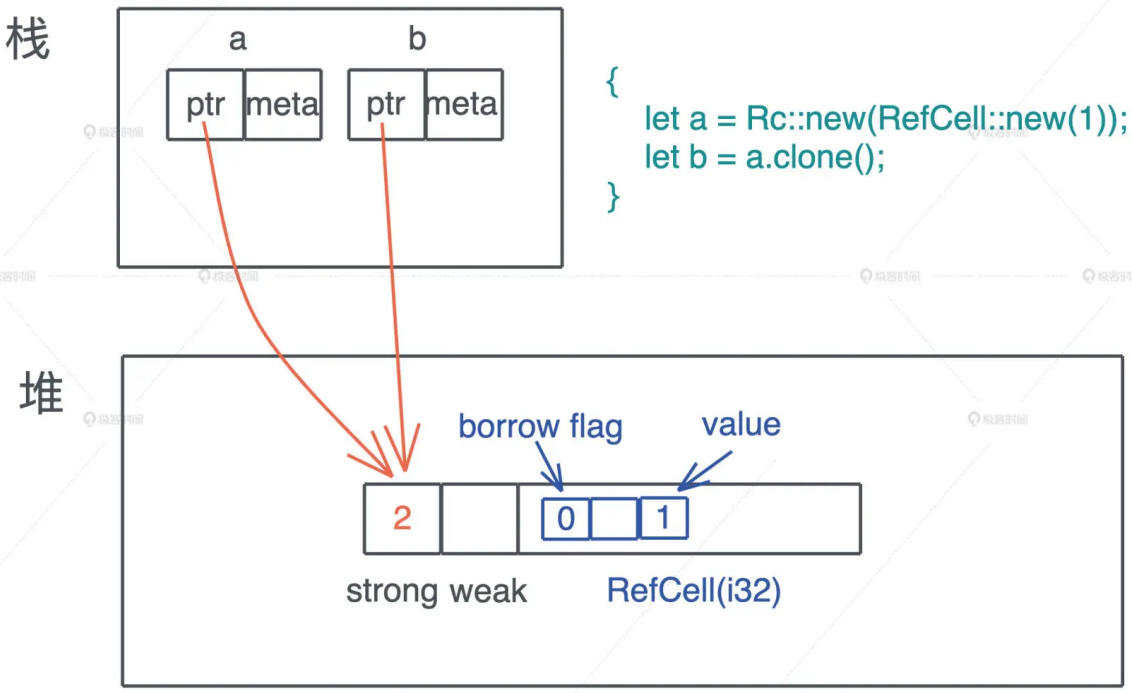
	使用方法	所有权检查
外部可变性	let mut 或者 &mut	编译时，如果不符合规则，产生编译错误
内部可变性	使用 Cell / RefCell	运行时，如果不符合规则，产生 panic

极客时间

实现可修改 DAG

好，现在我们对 RefCell 有一个直观的印象，看看如何使用它和 Rc 来让之前的 DAG 变得可修改。

首先数据结构的 downstream 需要 Rc 内部嵌套一个 RefCell，这样，就可以利用 RefCell 的内部可变性，来获得数据的可变借用了，同时 Rc 还允许值有多个所有者。



极客时间

完整的代码我放到这里了（ 代码 4 ）：

复制代码

```rust
use std::cell::RefCell;
use std::rc::Rc;

#[derive(Debug)]
struct Node {
    id: usize,
    // 使用 Rc<RefCell<T>> 让节点可以被修改
    downstream: Option<Rc<RefCell<Node>>>,
}

impl Node {
    pub fn new(id: usize) -> Self {
        Self {
            id,
            downstream: None,
        }
    }

    pub fn update_downstream(&mut self, downstream: Rc<RefCell<Node>>) {
        self.downstream = Some(downstream);
    }

    pub fn get_downstream(&self) -> Option<Rc<RefCell<Node>>> {
        self.downstream.as_ref().map(|v| v.clone())
    }
}

fn main() {
    let mut node1 = Node::new(1);
    let mut node2 = Node::new(2);
    let mut node3 = Node::new(3);
    let node4 = Node::new(4);

    node3.update_downstream(Rc::new(RefCell::new(node4)));
    node1.update_downstream(Rc::new(RefCell::new(node3)));
    node2.update_downstream(node1.get_downstream().unwrap());
    println!("node1: {:?}, node2: {:?}", node1, node2);

    let node5 = Node::new(5);
    let node3 = node1.get_downstream().unwrap();
    // 获得可变引用，来修改 downstream
    node3.borrow_mut().downstream = Some(Rc::new(RefCell::new(node5)));

    println!("node1: {:?}, node2: {:?}", node1, node2);
}

```

可以看到，通过使用 `Rc<RefCell<T>>` 这样的嵌套结构，我们的 DAG 也可以正常修改了。

Arc 和 Mutex/RwLock

我们用 Rc 和 RefCell 解决了 DAG 的问题，那么，开头提到的多个线程访问同一块内存的问题，是否也可以使用 Rc 来处理呢？

不行。因为 Rc 为了性能，使用的不是线程安全的引用计数器。因此，我们需要另一个引用计数的智能指针：Arc，它实现了线程安全的引用计数器。

Arc 内部的引用计数使用了 Atomic Usize ，而非普通的 usize。从名称上也可以感觉出来，Atomic Usize 是 usize 的原子类型，它使用了 CPU 的特殊指令，来保证多线程下的安全。如果你对原子类型感兴趣，可以看 std::sync::atomic 的文档。

Rust 实现两套不同的引用计数数据结构，完全是为了性能考虑，从这里我们也可以感受到 Rust 对性能的极致渴求。**如果不用跨线程访问，可以用效率非常高的 Rc；如果要跨线程访问，那么必须用 Arc。**

同样的，RefCell 也不是线程安全的，如果我们要在多线程中，使用内部可变性，Rust 提供了 Mutex 和 RwLock。

这两个数据结构你应该都不陌生，Mutex 是互斥量，获得互斥量的线程对数据独占访问，RwLock 是读写锁，获得写锁的线程对数据独占访问，但当没有写锁的时候，允许有多个读锁。读写锁的规则和 Rust 的借用规则非常类似，我们可以类比着学。

Mutex 和 RwLock 都用在多线程环境下，对共享数据访问的保护上。刚才中我们构建的 DAG 如果要用在多线程环境下，需要把 `Rc<RefCell<T>>` 替换为 `Arc<Mutex<T>>` 或者 `Arc<RwLock<T>>`。更多有关 Arc/Mutex/RwLock 的知识，我们会在并发篇详细介绍。

小结

我们对所有权有了更深入的了解，掌握了 Rc / Arc、RefCell / Mutex / RwLock 这些数据结构的用法。

如果想绕过“一个值只有一个所有者”的限制，我们可以使用 **Rc / Arc 这样带引用计数的智能指针**。其中，Rc 效率很高，但只能使用在单线程环境下；Arc 使用了原子结构，效率

略低，但可以安全使用在多线程环境下。

然而，Rc / Arc 是不可变的，如果想要修改内部的数据，**需要引入内部可变性**，在单线程环境下，可以在 Rc 内部使用 RefCell；在多线程环境下，可以使用 Arc 嵌套 Mutex 或者 RwLock 的方法。

你可以看这张表快速回顾：

访问方式		数据	不可变借用	可变借用
单一所有权		T	&T	&mut T
共享所有权	单线程	Rc<T>	&Rc<T>	无法得到可变借用
		Rc<RefCell<T>>	v.borrow()	v.borrow_mut()
	多线程	Arc<T>	&Arc<T>	无法得到可变借用
		Arc<Mutex<T>>	v.lock()	v.lock()
		Arc<RwLock<T>>	v.read()	v.write()

极客时间

思考题

1. 运行下面的代码，查看错误，并阅读 std::thread::spawn 的文档，找到问题的原因后，修改代码使其编译通过。

复制代码

```
fn main() {
  let arr = vec![1];

  std::thread::spawn(|| {
    println!("{:?}", arr);
  });
}
```

2. 你可以写一段代码，在 main() 函数里生成一个字符串，然后通过 `std::thread::spawn` 创建一个线程，让 main() 函数所在的主线程和新的线程共享这个字符串么？提示：使用 std::sync::Arc。
3. 我们看到了 Rc 的 clone() 方法的实现：

复制代码

```
fn clone(&self) -> Rc<T> {
    // 增加引用计数
    self.inner().inc_strong();
    // 通过 self.ptr 生成一个新的 Rc 结构
    Self::from_inner(self.ptr)
}
```

你有没有注意到，这个方法传入的参数是 `&self` ，是个不可变引用，然而它调用了 `self.inner().inc_strong()` ，光看函数名字，它用来增加 self 的引用计数，可是，为什么这里对 self 的不可变引用可以改变 self 的内部数据呢？

欢迎在留言区分享你的思考。恭喜你完成了 Rust 学习的第九次打卡，如果你觉得有收获，也欢迎分享给你身边的朋友，邀 TA 一起讨论。

参考资料

1. clone() 函数的实现源码
2. 最小权限原则
3. Rc::new() 的源码
4. Arc 内部的引用计数使用了 Atomic Usize
5. Atomic Usize 是 usize 的原子类型： std::sync::atomic 的文档
6. 内部可变性：除了 RefCell 之外，Rust 还提供了 Cell。如果你想对 RefCell 和 Cell 进一步了解，可以看 Rust 标准库里 cell 的文档。

分享给需要的人，Ta订阅后你可得 20 元现金奖励

赞 13 提建议

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 08 | 所有权：值的借用是如何工作的？

下一篇 10 | 生命周期：你创建的值究竟能活多久？

精选留言 (19)

写留言



lisiur

2021-09-10

老师您好，在代码3里，把内部作用域去掉会导致运行时错误，但是如果使用普通的借用方式像这样：

```rust
fn main() {...
```

展开

作者回复: 谢谢你这么有深度的思考！

编译器是一个不断进化的过程，在基本的规则下，它可以尽可能多地把常见的，并且合法的情况处理掉，不给开发者太多的负担。所以可以引用引入了是否活跃的概念，以及 NLL（https://rust-lang.github.io/rfcs/2094-nll.html）。

对于文中的内部可变性的例子，检查完全有赖正在执行的函数来确保。的确，对于这个例子，在编译期完全有可能把原本在作用域结束时 drop 的 v 提前到 v 使用完的地方：

```rust

use std::cell::RefCell;

fn main() {
    let data = RefCell::new(1);

    // 获得 RefCell 内部数据的可变借用
    let mut v = data.borrow_mut();
    *v += 1;
    // 提前 drop 可变引用
```

```
    drop(v);

    println!("data: {:?}", data.borrow());

    // 隐含的 drop
    // 原本 drop(v) 发生在这里
    drop(data);
}
```

这样可以避免额外的作用域。

但是，如果这样做，编译器就是在为某个特定的数据结构 RefCell，而非 Rust 的语法单元 &mut 来做优化了。从系统设计的角度，除非能找到一种很通用的方法（比如设计一个新的像 Send/Sync/Unpin 这样的 auto trait 对所有实现了这个 auto trait 的数据结构进行优化），否则，编译器需要知道哪个数据结构在做哪个操作的时候需要特殊处理。这就会让编译器本身的设计变得复杂。

个人意见，不代表编译器团队的思路 :)

10

千回百转无劫山
2021-09-10

1. 错误为线程借用的arr生命周期可能会长于main函数中的arr，简单处理的话把main中arr的所有权move到线程里即可，编译器对此有详尽的提示：
```rust
fn main() {
    let arr = vec![1];...
```
展开

作者回复: 赞！非常棒的回答。第 3 题其实不用看代码也可以尝试猜一下：当一个只读引用可以修改内部数据时，它一定是用了内部可变性。:)

一个实际使用的系统一定是符合二八定律的：可以用少量规则来满足 80%的应用场景。但总还有例外需要处理。所以编译期尽管能解决绝大多数的应用场景，但是解决不了的时候，还是需要运行期的检查来弥补。

这就涉及到如何权衡了。Rust 的选择是最小权限原则，你只有显式地要求（比如数据结构用 Arc

<T> 封装），才会进行额外的处理（维护引用计数）。其实这也是我们撰写软件系统时应该遵循的原则。

谈到滥用，最小权限原则恰恰是为了防止滥用。比如 mut，当你实际不需要的时候，编译器会报警。Rc/Arc 不必要的 clone，clippy（Rust 的 linting 工具）会提示。

如果「滥用」智能指针，并不会导致内存安全无法保证，比如你在不需要的时候使用 Arc，损失的是 atomic compare_and_swap 时的性能，但不会带来安全问题。

2 9



noisyes
2021-09-10

```
fn main() {
    /*let data = RefCell::new(1);

    let mut v = data.borrow_mut();
    *v += 1;...
```

展开

作者回复: 这就是运行期检查和编译期检查的区别。data.borrow_mut() 产生的 v 会一直活跃到作用域结束，而对于 &mut 编译器可以检查它是否活跃从而让我们撰写代码时不需要额外的作用域。你可以看看我和另一个同学关于这个问题的详细讨论。

4



Kerry
2021-09-12

课后思考题：

1. 生命周期问题，主线程可能比派生线程更早结束，导致派生线程引用了过期的值。

你也许会想到加上join强制派生线程先于主线程结束，以此解决编译问题。然而这是行不...

展开

作者回复: 1、2 正确！3 跟 unsafe 关系不大，跟 strong 是 Cell<usize> 实现了内部可变性有关。（当然，内部可变性底层的实现是 unsafe）

1 2



noisyes
2021-09-10

思考题：
1. fn main() {
let arr = vec![1];
let handler = std::thread::spawn(move || {
println!("{:?}", arr);...
展开

作者回复: 正确！

1



葡萄
2021-09-10

3. Rc类创建
Box::leak(box RcBox { strong: Cell::new(1), weak: Cell::new(1), value })
self.inner().inc_strong()
获得了Rc里 strong, 也就是Cell对象，Cell是内部可变的，使用set方法进行修改。
fn inc_strong(&self) {...
展开

作者回复: 非常正确！

1



Geek_364411
2021-09-10

1.
fn main() {
let arr = vec![1];

std::thread::spawn(move || {...
展开

作者回复: 1/2 正确！

3. 其实是想帮助我们回顾内部可变性。当一个不可变的引用可以修改内部状态时，它一定使用了某种内部可变性的方法。你的代码并没有解决 clone() 面临的问题，因为从接口看，你要修改的 dst 是一个 &mut T。clone() 第一个参数是 &self，不是 &mut self。

1

核桃
2021-09-16

代码1和4中留意一下struct 和impl都是Node，这样的写法看起来比较简洁。
另外代码中有一句
self.downstream.as_ref().map(|v| v.clone())
这里的map语法是什么意思？没太看懂，多谢了

展开

作者回复: downstream 是一个 Option<Node>，as_ref() 后变成 Option<&Nonde>，Option 有 map 方法，可以对里面的 Some(v) 的 v 进行 map 处理。可以看：https://doc.rust-lang.org/std/option/enum.Option.html#method.map

记事本
2021-09-15

x.borrow_mut() 和 &mut x 好像结果一样他们之间有什么区别吗？

作者回复: 文中讲过了，&mut x 编译时编译器可以检查是否违背所有权规则，borrow_mut() 是个函数，只有运行到这一刻，才知道是否违背所有权规则。

罗杰
2021-09-12

听取老师建议，读源码，看到 cell.rs 里面有很多的 unsafe，代码仔细阅读也没有那么难。

作者回复: 标准库里很多 unsafe 都是为了性能所做的妥协。读代码，首先看数据结构和接口，然后再看实现细节，很多标准库下的unsafe，实现细节并不复杂，如果看不懂，弄明白接口在干嘛也就可以。

0@1
2021-09-12

老师，能不能提前列出所有的课程目录，这样也好方便问问题， 有些问题可能需要在特定课程目录下问比较合适。

展开

编辑回复: 课程所有的目录？有啊，在课程首页就有一张目录长图。

最近留言区提的一些问题都要在后面才会学到。
一部分是因为Rust的设计是环环相扣的，所以之前的问题涉及后面的知识；一部分也是因为有些同学非常棒，一步步追问思考，学习进度也很不错，所以会问到目前的超纲内容。问了的，老师看到也都认真补充了。



Marvichov
2021-09-12

为啥要week == 0的时候才deallocate所有内存呢? 这样是不是不太高效?

```
fn drop(&mut self) {
    unsafe {...
```

展开

作者回复: 好问题。strong 为 0 时，ptr::drop_in_place 已经把大部分内存释放。但此时可以还有孤悬的 weak reference，它们还是可用的，但无法 upgrade。可以看这个代码：

```rust
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct Foo {
    bar: &'static str,
}

impl Drop for Foo {
    fn drop(&mut self) {
        println!("dropped!");
    }
}

fn main() {
```

```
    let foo = Rc::new(Foo { bar: "hello" });
    let weak_foo = Rc::downgrade(&foo);
    let other_weak_foo = Weak::clone(&weak_foo);

    drop(weak_foo); // 不会 drop
    drop(foo); // 打印 dropped!

    // 此时还有 weak ref 存在
    println!("other weak foo: {:?}", other_weak_foo);

    // 无法 upgrade
    assert!(other_weak_foo.upgrade().is_none());
}
```

playground: https://play.rust-lang.org/?version=stable&mode=debug&edition=2018&gist=ffd75a38877cfd78b6c66d0b13dc1d90

1



Arthur

2021-09-11

老师好，关于使用花括号提前结束生命周期这点有一点不明白

```rust
fn main() {
    let mut v = vec![1, 2, 3];
    {...
```

展开

作者回复: 在你用花括号括起 v 的那句，v 的所有权被转移给了 {} 作用域内部，并且由于没有人使用这个花括号的返回值，所以 v 被 drop 了。之后再用 v 就会报错：

```rust
fn main() {
    let mut v = vec![1, 2, 3];
    {
        v // 编译不通过
    };
    v.push(4);
}
```

而使用 v.push()，这里使用了 v 的可变引用，没有问题。






有铭
2021-09-11

今天这章我看完以后感觉有一股非常遗憾的情绪：
我原本以为rust真的用编译时检查解决了当年C++面临的那些问题。结果最后还是妥协撕开了一道口子开了后门，而且这个后门开的很复杂，我的感觉，这个特性，将来要么就是没什么人用，要么就是被人滥用。这个东西对人的自控要求太高了

展开

作者回复: 这个世界本就不纯粹，没有绝对的完美。所有的语言都会撕开一道口子，Haskell 也需要 IO Monad 和外界打交道，Erlang VM 也得忍着可能 crash VM 的风险引入 NIF (native function) 等等。不光语言如此，框架也如此，给你解决 80% 问题的利器，也保留让你解决剩下 20% 问题的灵活度。这还只是一个 Box:leak/Box::into_raw/ManuallyDrop，Rust 还有 unsafe 呢... 别着急下结论，让子弹飞一会。




Tyler
2021-09-10

1、在闭包前加move关键字。 编译器在编译时有错误提示并提供了一个解决的方法。
2、
use std::sync::Arc;

fn main() {...

展开

作者回复: 对！第 3 题和 unsafe 关系不大，和内部可变性有关。一个只读引用（&self）修改某个内部数据，那么这个内部数据一定使用了内部可变性。






Custer
2021-09-10

1.
fn main() {

```
let arr = vec![1];

std::thread::spawn(move || {...
```

展开

作者回复: 非常正确！



jin
2021-09-10

老师好，文章构造DAG时，是先构建了Node1，3，4的关系，再把2加进去。但让我直觉上难以接受的是，加进去的时候却用到了Node1，因为需要调用Rc.clone()增加引用计数。在实际的图论问题中，假设我们要插入点2，找到了点3，难道还要O(n)遍历所有点，从而找到点2的reference吗？感觉这个确实有点限制灵活性了，请问您怎么看呢？

展开

作者回复: 这是这个例子的写法问题，它只是为了展示 Rc / RefCell。如果要真写一个 DAG，可以看 daggy 这个项目，这是它的用法：https://github.com/mitchmindtree/daggy/blob/master/tests/add_edges.rs



Christian
2021-09-10

1. spawn 要求闭包具有静态生命周期。但传入的闭包捕获了 arr 的引用，生命周期不会超出 main 函数。修复：

```rust
fn main() {
    let arr = vec![1];...
```

展开

作者回复: 正确！不过第 2 题在线程里的闭包没有必要。



夏洛克Moriaty
2021-09-10

终于理解了Rc的本质了，以前只是知道实现二叉树这种数据结构必须用Rc，因为他允许有

多个所有者。但是不理解或者说不知道为什么要有内部可变性和运行时检查这个东西，今天看了Box::leak() 机制的解释终于明白了。

展开

作者回复: 👍

1